

✓ Copyright 2019 The TensorFlow Authors.

> Licensed under the Apache License, Version 2.0 (the "License");

[Show code](#)

## ✓ Using TensorBoard in Notebooks



[View on TensorFlow.org](#)



[Run in Google Colab](#)



[View source on GitHub](#)



[Download notebook](#)

TensorBoard can be used directly within notebook experiences such as [Colab](#) and [Jupyter](#). This can be helpful for sharing results, integrating TensorBoard into existing workflows, and using TensorBoard without installing anything locally.

## ✓ Setup

Start by installing TF 2.0 and loading the TensorBoard notebook extension:

**For Jupyter users:** If you've installed Jupyter and TensorBoard into the same virtualenv, then you should be good to go. If you're using a more complicated setup, like a global Jupyter installation and kernels for different Conda/virtualenv environments, then you must ensure that the `tensorboard` binary is on your `PATH` inside the Jupyter notebook context. One way to do this is to modify the `kernel_spec` to prepend the environment's `bin` directory to `PATH`, [as described here](#).

**For Docker users:** In case you are running a [Docker](#) image of [Jupyter Notebook server using TensorFlow's nightly](#), it is necessary to expose not only the notebook's port, but the TensorBoard's port. Thus, run the container with the following command:

```
docker run -it -p 8888:8888 -p 6006:6006 \
tensorflow/tensorflow:nightly-py3-jupyter
```

where the `-p 6006` is the default port of TensorBoard. This will allocate a port for you to run one TensorBoard instance. To have concurrent instances, it is necessary to allocate more ports. Also, pass `--bind_all` to `%tensorboard` to expose the port outside the container.

```
# Load the TensorBoard notebook extension
%load_ext tensorboard
```

Import TensorFlow, datetime, and os:

```
import tensorflow as tf
import datetime, os
```

## ✓ TensorBoard in notebooks

Download the [FashionMNIST](#) dataset and scale it:

```
fashion_mnist = tf.keras.datasets.fashion_mnist
```

```
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ub
29515/29515 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ub
26421880/26421880 ————— 1s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-uby
5148/5148 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-uby
4422102/4422102 ————— 1s 0us/step
```

Create a very simple model:

```
def create_model():
    return tf.keras.models.Sequential([
        tf.keras.layers.Flatten(input_shape=(28, 28), name='layers_flatten'),
        tf.keras.layers.Dense(512, activation='relu', name='layers_dense'),
        tf.keras.layers.Dropout(0.2, name='layers_dropout'),
        tf.keras.layers.Dense(10, activation='softmax', name='layers_dense_2')
    ])
```

Train the model using Keras and the TensorBoard callback:

```
def train_model():

    model = create_model()
    model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

    logdir = os.path.join("logs", datetime.datetime.now().strftime("%Y%m%d-%H%M%S"))
    tensorboard_callback = tf.keras.callbacks.TensorBoard(logdir, histogram_freq=1)

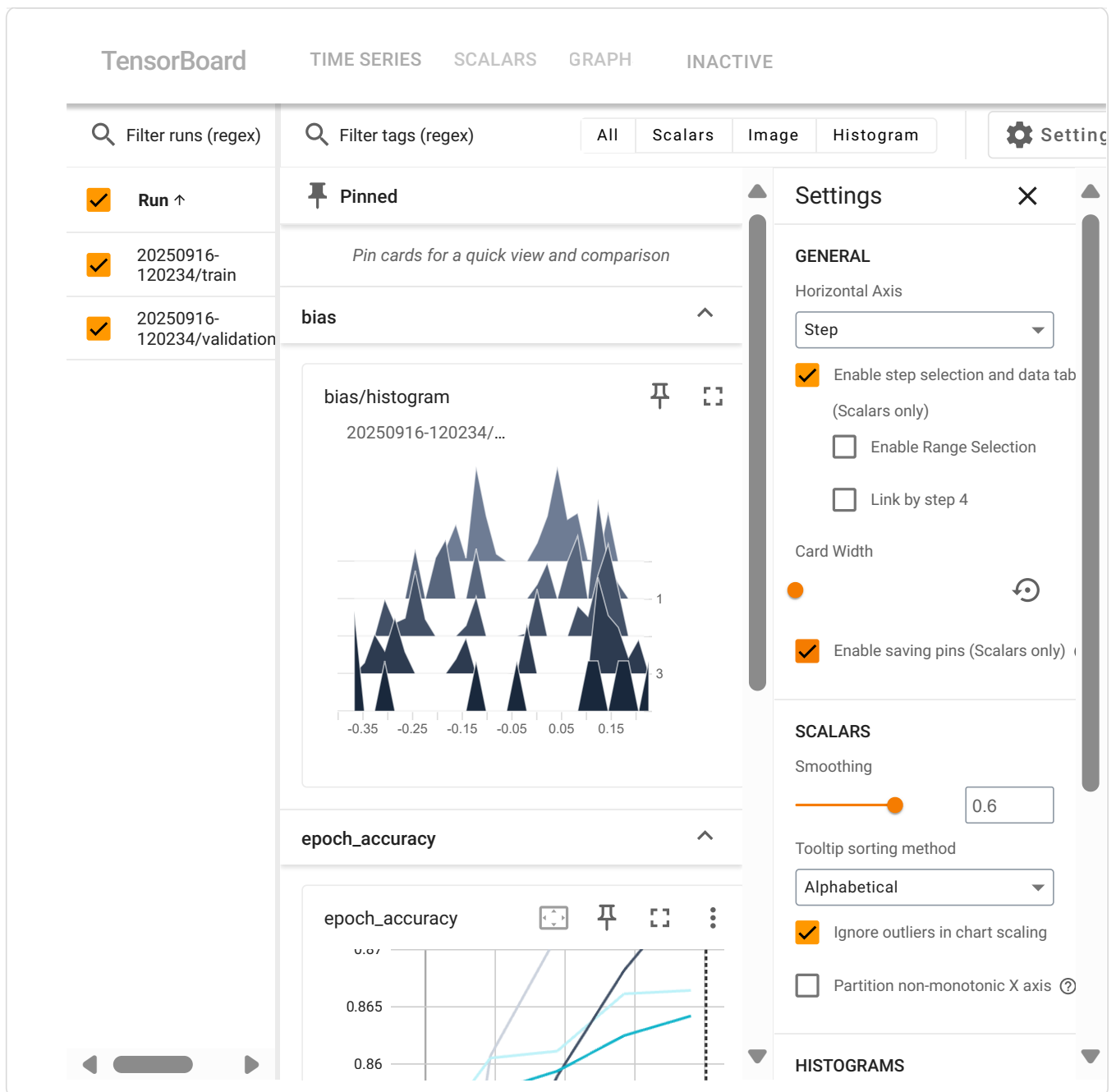
    model.fit(x=x_train,
              y=y_train,
              epochs=5,
              validation_data=(x_test, y_test),
              callbacks=[tensorboard_callback])

train_model()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not
super().__init__(**kwargs)
Epoch 1/5
1875/1875 ————— 17s 8ms/step - accuracy: 0.7826 - loss: 0.6121 - val_accuracy: 0.8526 -
Epoch 2/5
1875/1875 ————— 16s 9ms/step - accuracy: 0.8569 - loss: 0.3921 - val_accuracy: 0.8605 -
Epoch 3/5
1875/1875 ————— 20s 8ms/step - accuracy: 0.8690 - loss: 0.3552 - val_accuracy: 0.8611 -
Epoch 4/5
1875/1875 ————— 15s 8ms/step - accuracy: 0.8764 - loss: 0.3314 - val_accuracy: 0.8661 -
Epoch 5/5
1875/1875 ————— 16s 8ms/step - accuracy: 0.8857 - loss: 0.3123 - val_accuracy: 0.8664 -
```

Start TensorBoard within the notebook using [magics](#):

```
%tensorboard --logdir logs
```



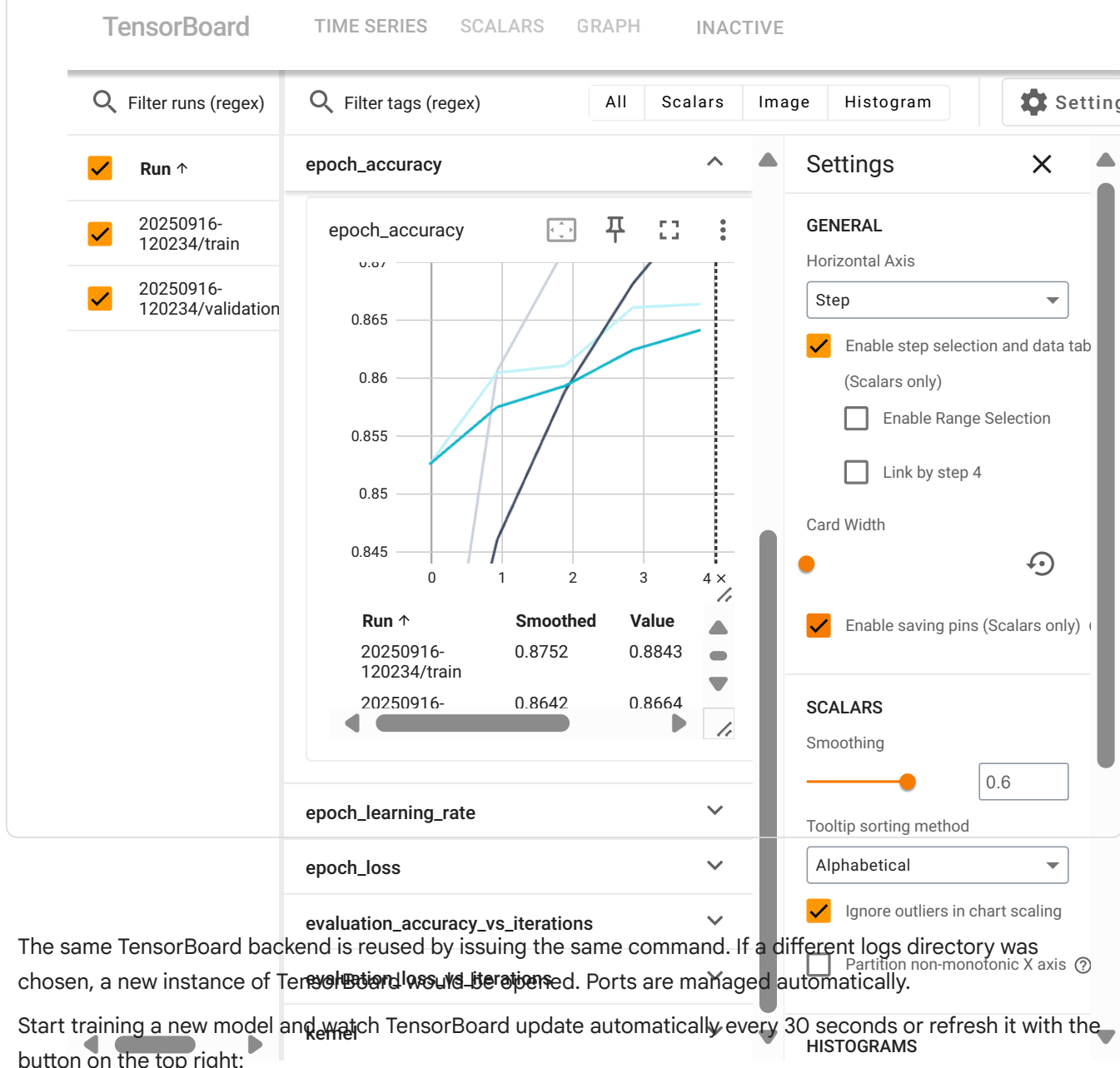
You can now view dashboards such as **Time Series**, **Graphs**, **Distributions**, and others. Some dashboards are not available yet in Colab (such as the profile plugin).

The `%tensorboard` magic has exactly the same format as the TensorBoard command line invocation, but with a `%`-sign in front of it.

You can also start TensorBoard before training to monitor it in progress:

```
%tensorboard --logdir logs
```

Reusing TensorBoard on port 6006 (pid 921), started 0:00:03 ago. (Use '!kill 921' to kill it.)



The same TensorBoard backend is reused by issuing the same command. If a different logs directory was chosen, a new instance of TensorBoard would be opened. Ports are managed automatically.

Start training a new model and watch TensorBoard update automatically every 30 seconds or refresh it with the button on the top right:

```
train_model()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/resizing/flatten.py:37: UserWarning: Do not
super().__init__(**kwargs)
Epoch 1/5
1875/1875 ————— 17s 9ms/step - accuracy: 0.7825 - loss: 0.6191 - val_accuracy: 0.8425 -
Epoch 2/5
1875/1875 ————— 20s 8ms/step - accuracy: 0.8568 - loss: 0.3924 - val_accuracy: 0.8574 -
Epoch 3/5
1875/1875 ————— 21s 8ms/step - accuracy: 0.8721 - loss: 0.3483 - val_accuracy: 0.8641 -
Epoch 4/5
1875/1875 ————— 15s 8ms/step - accuracy: 0.8767 - loss: 0.3287 - val_accuracy: 0.8610 -
Epoch 5/5
1875/1875 ————— 15s 8ms/step - accuracy: 0.8824 - loss: 0.3165 - val_accuracy: 0.8712 -
```

You can use the `tensorboard.notebook` APIs for a bit more control:

```
from tensorboard import notebook
notebook.list() # View open TensorBoard instances
```

Known TensorBoard instances:

```
# Control TensorBoard display. If no port is provided,  
# the most recently launched TensorBoard is used  
notebook.display(port=6006, height=1000)
```

Selecting TensorBoard with logdir logs (started 0:02:06 ago; port 6006, pid 921).

